Functions

- Functions start with the **def** keyword, which acts as a header to signal the name of the function. The body is indented and contains the code. Def my_function():
- You must call the function in order for it to run after you have defined it.
- **Parameters** are what the function expects when it is called, parameters can be any kind of type that a variable can be, included inside the parentheses of the definition, my_function(num1,num2)
- Parameters are in the function definition, while arguments used when the function is called
- **Local scope** refers to the fact that a variable within a function is true for just the function and not the rest of the code. This always applies **except for lists**.
- The **return** keyword will save the answer of the function to later be called outside of the function, my_function(num1,num2): >>return(num1+num2)

Dictionaries

- **Dictionaries** are similar to lists, but to find a value you must enter a key value rather than an index. Like lists they are mutable, dynamic, and can be nested. Unlike lists, dictionaries use curly brackets and colons to separate the key values. Accessing a value is similar, print(dictionary[‘key_value”]). Key values can only be immutable types, so not lists or dictionaries.
- The **keys** method of a dictionary generates a sequence of all the keys in the dictionary, similarly to how the range function generates a sequence of integers, for key in dict.keys():
- The **values** method is the same as keys, but for values.
- The **items** method combines both the value and the key into one pair, for key, value in dict.items():
- The **in** operator can be used on dictionaries to search for keys, if value in dict.
- The **del** operator deletes both the key and a value in a dictionary , del dict[key]

Numpy + matplotlib modules

- %matplotinline may be necessary for plotting in jupiter notebook
- **Import** numpy as np, and matplotlib.pyplot as plt
- **Plt.axis** sets the range of the graph with 4 arguments for the min/max of the x/y range, plt.axis((-1,1,-1,1))

- **Plt.plot** plots the first 2 arguments given, typically lists, and the 3rd argument relates to the color or shape of the plots, where you can give ROYGBIV colors and - as a full line, - - as a dotted line, d as diamonds, etc, plt.plot(x1,y1,'b- -').
- **plt.x/ylabel** labels the x/y axis with the given string, plt.xlabel('time'). This should be called after plt.plot
- **plt.x/yscale** works similar to label, but adjusts the scale of the graph to something other than the default linear, plt.yscale('log')
- **Plt.show** prints the graph
- **Plt.title** adds a title, plt.title('title')
- **Plt.legend** adds a legend to the graph to label multiple plotted lines. This is useful when plotting multiple datasets at once, plt.legend().
- Labels for the legend are typically included as an argument in the plot call, then displayed with legend, plt.plot(x,y,label='dataset1').
- **Plt.scatter** creates a scatter plot instead of a line graph. This is useful when showing relationships between two variables rather than continuous functions, plt.scatter(x,y).
- **Plt.grid** adds a grid to the graph which can make values easier to read, plt.grid().
- **Plt.figure** creates a new figure window and can control the size of the graph, plt.figure(figsize=(6,4)).
- Subplots allow multiple graphs to exist within the same figure. This is useful for comparing graphs side by side, plt.subplot(rows,columns,position).
- For example, plt.subplot(2,1,1) creates the first graph in a layout of 2 rows and 1 column.
- **Plt.tight_layout** automatically adjusts spacing between subplots so labels and titles do not overlap, plt.tight_layout().

Numpy arrays

- **Np.array** is effectively an enhanced list, however arrays do not use any of the list methods to manipulate them, np.array([ ])
- **Np.append** works by making a new copy of the previous array with the new value at the end, but needs different arguments than the list method, needing the first argument as which array you're appending to, then what is being appended, np.append(array,'value')
- Arrays must be homogenous with types of data, i.e. all strings, integers, etc.
- To replace the original array, you must equal the array to the append method, ie array= np.append(array,'value')
- Accessing values in arrays works the same as in lists, simply call the index at which the value is located, array[0]

- Arrays can directly handle mathematical calculations, where it will be applied to all values of the array, array * 5
- Arrays can be filled out with lists, array(list)
- Arrays have many many math focused methods, such as min,max,sum,std, etc.
- **array.arange** creates a range of values from the min to the max, with a step included, does not include the max value, ary1.arrange(1,101,2)
- **.loadtxt** loads a data set using numpy, np.loadtxt('data_name')
- To easily mask a df, simply put a numerical boolean statement inside a df bracket, which will produce a similar data frame with real values (not just t or f) that meet the boolean, mask= df[df[: (row) , 7 (column)] <=30]

Panda module

- **Import** pandas is used to import the module, where pandas is like an excel sheet, displaying different columns simultaneously, import pandas as pd.
- **.read_json/_csv** is used to read the data from a json file, then put it into the panda structure, data= pd.read_csv('file', skiprows=x, delimiter=',')
- **.head/.tail** is used to give just the top/bottom few lines of the data, data.head()
- .**describe** collects various statistical analyses of the data like the mean, median, etc, data.describe()
- Information from a single column can be displayed, data['age']. This is used in conjunction with .plot, plotting the specific column of data, as well as .hist or any other type of graph to produce it, data['age'].plot.hist()
- **.keys** displays all of the headers of the columns, data.keys()
- **.scatter** creates a scatter plot, but needs x and y values, data.plot.scatter(x='age', y='str')
- **.iloc** accesses a value in the dataset by its indices or position data[column_1].iloc[row, column]. Include 2 arguments.
- **Slice** or : can be used to grab every single column/row with a potential non inclusive endpoint,
- **.loc** accesses a value in the dataset by its label or name, data[column_1].loc['a']. Loc is generally used for a row, data.loc['row_name'].
- Entire columns can be grabbed with data['column']
- **Dropna** is used to drop NaN values from a dataset, data.dropna(axis="columns", how="all"), data.dropna()
- **.columns/.rows** can be used to modify the headers of the rows/columns, data.columns=data.iloc[0,:]
- **.drop** is used to drop rows/columns from the set, data.drop(['row_name', etc])

Curve Fit

- From scipy.optimize import curve_fit
- **Curve_fit** plots a line of best fit with 3 or more arguments, the first being a function of the type of line that you're working with, a set of x values, a set of y values, and other variables if necessary, like c. output, covariance matrix= curve_fit(linear_function, x, y).
- Curve fit needs two arguments to be called, the second of which is a covariance matrix which can be safely ignored as long as it is called.
- Curve fit needs a function as its first argument, which is simply the type of line that is being used, def linear_function(m,x,b) return m*x+b

I, Cameron Korson, commit to recognizing the privilege of pursuing a college education. In appreciation of this opportunity and the efforts of those who helped make it possible, I pledge to apply myself fully in my studies, complete and submit work that reflects my own efforts, and properly acknowledge any sources or assistance used. I affirm that I understand and will adhere to Michigan State University's policies regarding academic integrity, including rules on plagiarism and cheating.

CMSE 202

System Navigation & Information

clear: Clears all previous text from your terminal screen to give you a blank slate.

pwd: Prints the full path of the folder you are currently working inside.

ls: Lists all the visible files and folders inside your current directory.

cd [directory]: Moves your terminal focus into the specific folder path you type.

cd ..: Jumps upward into the parent folder of your current directory.

cd .: Keeps you exactly where you are, as a single dot represents the current folder.

cd ~ (BEST): Instantly takes you back to your user profile's main home directory.

cd -: Toggles back to the previous directory you were working in before your last move.

Creating & Managing Files

- **touch [new file]**: Creates an empty file with the name and extension you provide.
- **cp**: The general command used to duplicate files or folders from one place to another.
- **cp [file] [new file]**: Makes an exact duplicate copy of a file under a brand new name.
- **cp -r [folder] [new folder]**: Copies an entire folder and all of its contents recursively to a new location.
- **mv**: The general command used to physically relocate or rename your items.
- **mv [path to file/folder] [new path to file/folder]**: Moves an item to a new directory or changes its name right where it is.

Deleting Items

- **rm**: The foundational command used to permanently delete items from your system.
- **rm [file]**: Deletes a specific file instantly without sending it to a recycle bin.
- **rm -r [folder]**: Recursively forces the permanent deletion of a folder and everything inside it.

Viewing File Content

- **less [file]**: Opens a file for reading, letting you scroll up and down using the arrow keys.
- **more [file]**: Displays a file page-by-page, advancing forward whenever you press the spacebar.

CURL

curl: A command-line tool used to transfer data to or from a network server.

-o: A specific flag that directs the downloaded data into a local file instead of printing it to the terminal screen.

Syntax: The required format is `curl -o filename.ext URL`, where the custom name must always be typed immediately after the flag, followed by the web address.

OBJECTS & OOP

Defining a Class

- **`class` Keyword:** You define a class using the `class` keyword followed by its name and a colon. It acts as a blueprint for creating objects, similar to how `def` sets up a function.
- **Evolution:** Start with a basic class description containing just a few attributes and methods. Test this simple version first, then add more features as your project grows.

The `self` Keyword

- **Reference:** The `self` parameter represents the specific object you are currently working with. Python requires it as the very first argument in every class method so the function knows which object to modify.
- **Attributes:** These are variables permanently attached to a class. To create or use them inside a method, you must prefix them with `self.` (e.g., `self.name = 'Milka'`) so Python doesn't mistake them for temporary variables.
- **Calling Methods:** To call one method from inside another method within the same class, you must prefix the call with `self.` (e.g., `self.get_name()`).

The Constructor (`__init__`)

- **Special Method:** The `__init__` function is a built-in method reserved by Python, easily recognized by its double underscores.
- **Automatic Execution:** You never call `__init__` manually. Python runs it automatically behind the scenes the exact moment a new object is created.
- **Purpose:** Known as a "constructor," it takes your initial input parameters (like animal type or nickname) and assigns them to the object's starting attributes so they are ready to use.

Naming Standard Methods

- **Setters:** Methods built to update or change an existing attribute's value (e.g., `set_name`, `set_kind`). They take a new value as an argument and assign it to a `self.` variable.

- **Getters:** Methods built purely to look up and return an attribute's value (e.g., `get_name`, `get_kind`). They don't change data and usually require no inputs other than `self`.